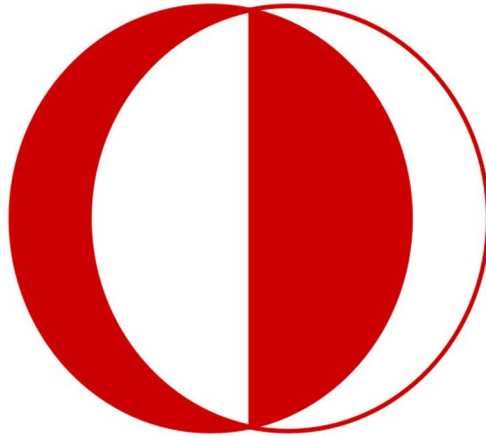


19/06/2026



CE4011
Spring 2025/26 - TERM PROJECT

**GUI- Based Static and Modal Postprocessing Extension of a 2D
Frame–Truss Direct Stiffness Solver**

By:

Imran Shahriar

ID: 2735835

Faculty of Civil Engineering

Graduate School of Natural and Applied Sciences

Middle East Technical University

To:

Prof. Özgür Kurç

Faculty of Civil Engineering

Graduate School of Natural and Applied Sciences

Middle East Technical University

Project repository: <https://github.com/ImranMETU/CE4011-Structural-Analysis-GUI>

Video Presentation Link:

<https://drive.google.com/drive/folders/1LKxH3PTDTnRRohcugyNjN5usN5ejap3>

Python environment: CE4011, Python 3.11

1. Introduction, Scope, and Development Background

This project presents a GUI-based structural analysis and postprocessing program for two-dimensional frame–truss systems. The software extends the Direct Stiffness Method solver developed through the earlier CE4011 assignments into an educational analysis package with model input, linear static analysis, modal analysis, graphical postprocessing, result tables, and verification tools. The program is intended for structural-analysis learning and software-development demonstration, not as a replacement for commercial design software.

The motivation for the project was to move beyond a backend-only solver workflow. In the earlier assignments, structural models were mainly created through files or scripts, and the user had to inspect numerical outputs directly. The final project adds a graphical workflow in which users can define or load a model, inspect its geometry, run static or modal analysis, view deformed shapes and internal force diagrams, open result tables, and export outputs. The interface was implemented using Tkinter and Matplotlib, while the computational backend remains separated from the GUI.

The main computational extension of the final project is modal analysis. The modal module assembles lumped mass data, handles massive and massless degrees of freedom, performs static condensation where required, solves the generalized eigenvalue problem, and recovers full mode shapes. The resulting outputs include natural frequencies, periods, mode shapes, modal participation factors, effective modal masses, and modal mass ratios. These modal quantities also support additional educational response-history and response-spectrum postprocessing tools.

2. Software Architecture and Class Organization

The program is organized as a layered modular desktop application. Architecturally, it can be described as a modular monolith with an object-oriented structural domain model and a largely functional numerical core. The design separates model definition, analysis, postprocessing, visualization, and GUI interaction so that the same backend can be used by scripts, automated tests, verification examples, and the Tkinter interface.

Table 1. Main software layers

Layer	Main packages	Role
Structural model	src/model, src/units	Nodes, materials, sections, frame/truss elements, structure container, and unit metadata
Input and application state	src/io, src/gui, src/generators	File loading, form input, model editing, saving, and generated frame models

Analysis	src/analysis, matrix library	Static/modal calculations, mass assembly, damping, response utilities, and axis transformation
Postprocessing and visualization	src/postprocessing, src/visualization	Result extraction, tables, deformed shapes, force diagrams, modal plots, drift, and response plots
Verification	scripts, tests, verification	Smoke scripts, automated tests, regression tests, and verification outputs

The central backend object is **Structure**, which aggregates nodes, materials, sections, elements, loads, prescribed displacements, modal masses, and analysis data. **Node**, **Material**, and **Section** store the basic model properties, while the abstract **Element** class defines common element behavior. The two main element implementations are **FrameElement** and **TrussElement**; frame elements include axial and flexural behavior, while truss elements are axial-only. This gives the main inheritance structure: **Element** → **FrameElement**, **TrussElement**.

This hierarchy allows the global structure to assemble stiffness and recover element forces polymorphically, without hard coding a separate assembly path for each element type. The numerical infrastructure inherited from earlier assignments also provides matrix and solver abstractions, including sparse matrix storage and conjugate-gradient solution.

The GUI is not the solver's data model. **StaticAnalysisApp** acts as the main GUI controller, while **ModelBuilder** stores editable GUI-side records and emits a canonical model dictionary. JSON, XML, text-deck input, form input, and the parametric frame generator all produce the same model format. Once converted into a **Structure**, the model follows the same static, modal, postprocessing, plotting, and table-generation pipeline.

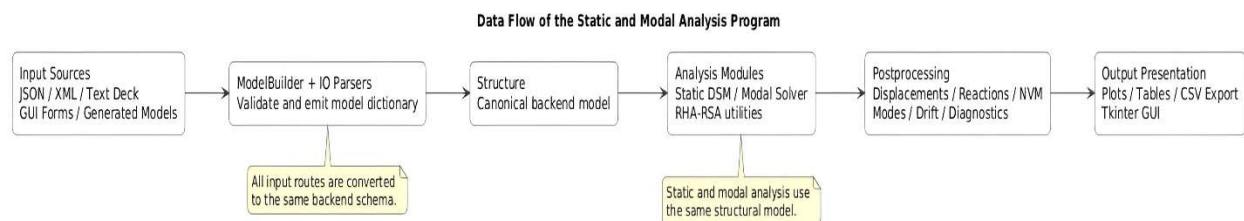


Figure 1. Data Flow Diagram

The main architectural strength is modular separation: solvers do not create GUI widgets, plotting functions do not solve equations, input loaders do not draw figures, and postprocessing functions do not modify stiffness assembly. This separation made automated testing and script-based verification easier. The main weakness is that **StaticAnalysisApp** has grown into a large controller responsible for loading, saving, menu callbacks, analysis execution, plotting, and table coordination. A future refactoring could split these responsibilities into smaller models, analysis, plotting, persistence, and result-view controllers.

A late-stage import-stability correction was also made. Some package initializers originally imported optional modules too early, creating circular-import risk for standalone scripts. The initializers were made lightweight, and import-stability regression tests were added. This changed software organization only; it did not alter stiffness assembly, modal analysis, postprocessing formulas, or numerical results.

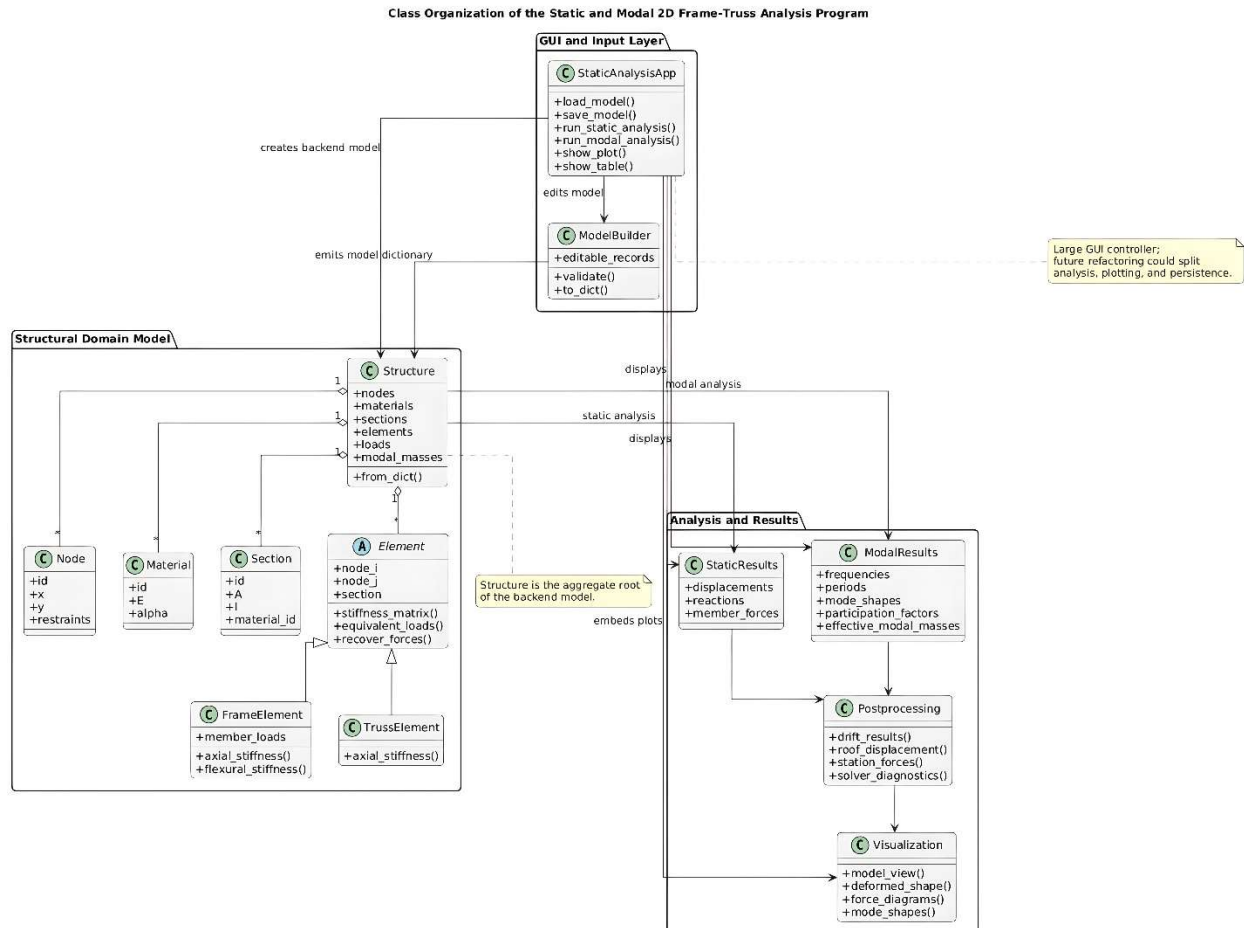


Figure 2. Combined architecture and class organization.

3. Numerical Methods

3.1 Linear Static Direct Stiffness Method

The static solver follows the Direct Stiffness Method for two-dimensional frame-truss structures. Each element forms a local stiffness matrix, transforms it to global coordinates, and contributes to the assembled global stiffness matrix. The governing equilibrium equation is $Ku = F$, where (K) is the global stiffness matrix, (u) is the displacement vector, and (F) is the load vector. After

support restraints and prescribed displacement conditions are applied, the reduced active-DOF system is solved, and the full displacement vector is recovered.

The static workflow includes nodal loads, equivalent nodal loads from supported member loads, thermal loading, and support settlements. After solving for displacements, the program recovers support reactions and local member-end forces. These results are then used for deformed-shape plotting, internal-force diagrams, station-force output, drift, and roof-displacement calculations.

3.2 Modal Analysis and Massless DOF Condensation

The main numerical extension of the project is modal analysis. The solver combines the structural stiffness matrix with a lumped mass matrix assembled from user-defined, generated, or companion-file modal masses. In lateral frame models, mass is often assigned to translational floor DOFs, while rotations and some vertical DOFs may remain massless.

Because massless DOFs can make the generalized mass matrix singular, the solver separates free DOFs into massive and massless sets. The massless DOFs are statically condensed, giving the reduced eigenvalue problem $K_c \varphi = \lambda M_c \varphi$ where (K_c) and (M_c) are the condensed stiffness and mass matrices, (φ) is the modal vector, and (λ) is the eigenvalue. The eigenvalue is related to the circular natural frequency by $\lambda = \omega^2$ and the period is $T = 2\pi/\omega$

After solving the condensed eigenproblem, the full free-DOF mode shapes are recovered. The modal postprocessor reports frequencies, periods, mode shapes, participation factors, effective modal masses, modal mass ratios, cumulative mass ratios, and DOF classifications. These outputs allow the modal solver to be checked through both numerical comparisons and physical mode-shape behavior.

The modal solver also supports experimental linear modal response-history and response-spectrum postprocessing utilities. These use the computed modal properties to process ground-motion records, generate elastic response spectra, estimate modal response histories, and combine peak modal responses using ABSSUM, SRSS, and CQC. These tools are educational modal postprocessing add-ons, not nonlinear response-history analysis or full commercial seismic design workflows.

4. User Interface and Input Workflow

A simplified SAP2000-like graphical interface was developed using Tkinter and Matplotlib. The interface provides a complete analysis workflow without requiring the user to edit backend data structures or call solver functions directly. The GUI is organized through the [File](#), [Define](#), [Analyze](#), [Display](#), [Tables](#), [Select](#), and [Help](#) menus.

The program supports five model input routes: JSON files, XML files, text-deck files, form-based GUI input, and generated parametric frame models. All input routes are converted into the same backend model dictionary before being passed to the **Structure** constructor. This common scheme allows models created from files, forms, or generators to use the same analysis and postprocessing pipeline.

The main workflow is load or define a model, inspect the model view, run static or modal analysis, view plots and tables, and export results. The **File** menu handles model loading, saving, and generated-model access, including companion modal mass files where applicable. The **Define** dialogs update the GUI-side **ModelBuilder**, which emits backend-compatible model data. This separates editable GUI records from the validated engineering objects used by the solver.

The **Analyze** menu runs static analysis, modal analysis, and optional modal response utilities. The **Display** and **Tables** menus provide graphical and numerical access to deformed shapes, internal-force diagrams, mode shapes, drift profiles, response plots, displacements, reactions, member-end forces, modal properties, station forces, roof displacement, and solver diagnostics. Result tables can also be exported as CSV files.

The model view is available before analysis and shows nodes, elements, supports, labels, loads, settlements, modal masses, and axis-offset indicators. This helps detect modeling errors before running the solver. Interactive selection tools allow the user to inspect nodes and elements in larger generated frame models.

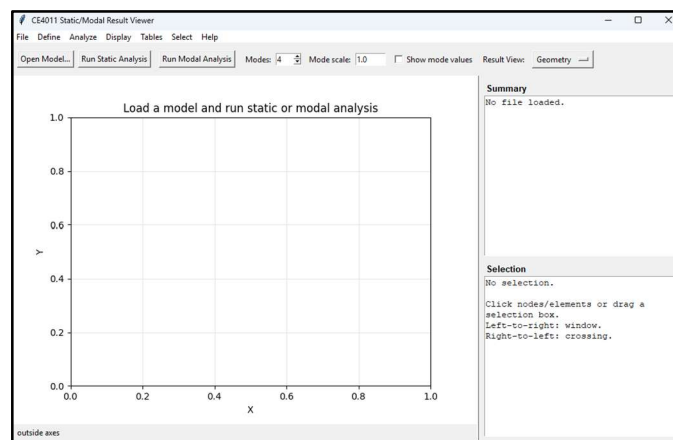


Figure 3. GUI view.

The text-deck format was retained as a compact reproducible input route for tests and verification examples. Its full syntax is documented separately in the user manual or appendix.

5. Analysis and Postprocessing Features

The postprocessing layer converts solver results into engineering outputs that can be inspected through GUI plots, result tables, CSV files, and verification scripts.

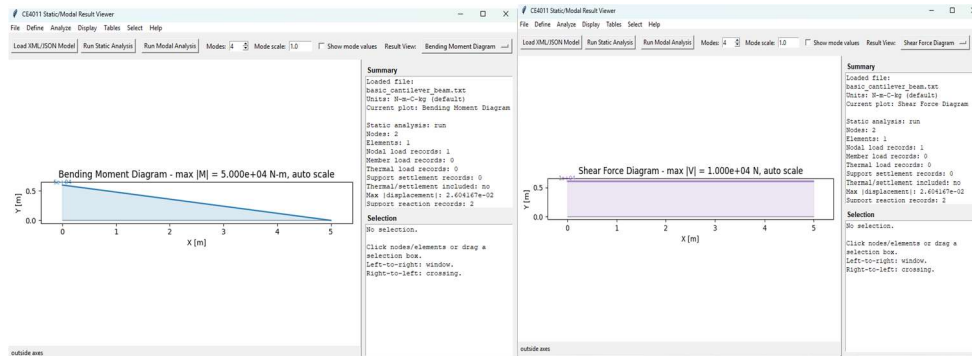


Figure 4. Static result visualization

After static analysis, the program reports displacement, reaction, and member-force results through tables and plots. The static deformed-shape plot uses visual magnification so that small physical displacements remain visible, while actual displacement values remain available in the tables. Internal-force diagrams are drawn on the actual two-dimensional member geometry using local member tangents and normals. A display-convention setting controls the plotted side or sign of force diagrams without changing the underlying numerical force values.

Internal forces are available in both member-end and station-based forms. Member-end force tables provide compact end results, while station-force tables report axial force (N), shear force (V), and bending moment (M) at multiple points along each member. This is useful for interpreting UDL and point-load effects along an element.

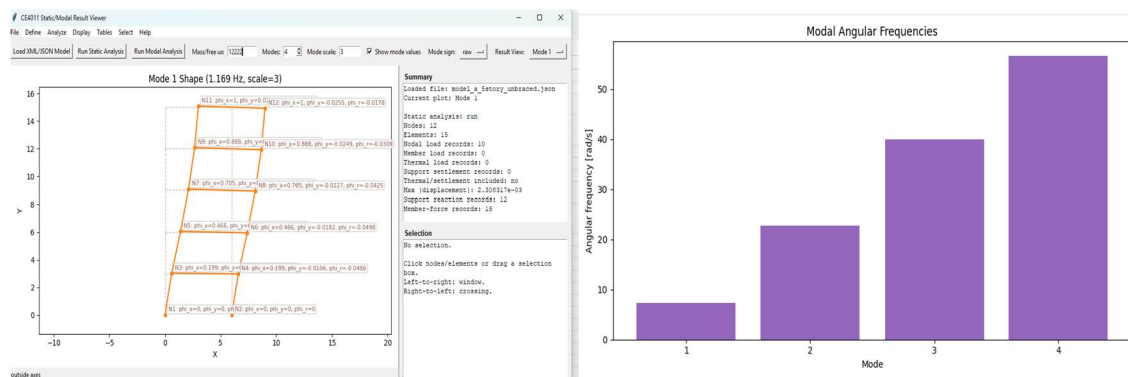


Figure 5. Modal result visualization (Mode 1 and Natural Frequencies).

Modal postprocessing reports eigenvalues, circular frequencies, frequencies in hertz, periods, mode shapes, participation factors, effective modal masses, modal mass ratios, and cumulative

modal mass ratios. Since eigenvector sign is arbitrary, the GUI includes mode-shape display sign handling to make plots easier to compare visually without changing the modal solution.

6. Verification and Validation

6.1 Automated Tests and GUI Backend Verification

The final implementation was checked using automated unit tests, regression tests, plotting tests, GUI-backend consistency tests, import-stability tests, and smoke scripts.. The final automated suite contained 347 collected pytest cases covering static analysis, modal analysis, mass handling, RHA/RSA utilities, input validation, postprocessing, plotting, GUI-backend consistency, model generation, saving/reopening, import stability, and regression examples. These were supplemented by a 13-case GUI-backend verification manifest, final core smoke testing, and external/reference validation using Ftool, CE586 examples, and Autodesk Robot.

The GUI backend verification cases include Assignment 4-style thermal and support-settlement examples, JSON/text/XML input routes, and one expected-error legacy settlement case. Additional regression tests were added for force-diagram geometry, deformed-shape autoscaling, modal mass handling, import stability, model saving, unit labels, and frontend/backend result consistency. A late-stage circular-import issue found in `generate_proposal_models.py` was fixed by making package initializers lightweight; this was an import-architecture correction only and did not change the structural formulations. The tests performed are in the appendix.

6.2 Static Validation

Static validation was performed using simple hand-checkable cases and Ftool software packages. The static solver was checked for displacement recovery, reaction equilibrium, member-end force recovery, internal-force diagram behavior, support-settlement handling, thermal loading, member point loads, and full-span uniformly distributed loads.

For the generated five-story unbraced frame, the total applied lateral load is 50kN, applied as ten nodal loads of 5kN. The sum of horizontal base reactions was checked against this applied load. The maximum computed static displacement was approximately 2.37 mm, while the Ftool result was approximately 2.30 mm. This agreement confirmed that the generated frame stiffness, loading, and boundary conditions were interpreted consistently. Internal-force diagrams were also checked against beam and frame cases; diagram display signs were treated as visualization conventions and did not alter the recovered numerical member forces.

6.3 Modal Validation

The modal solver was validated using CE586 Lecture 6 modal-analysis examples. These examples directly test lumped mass assembly, massless DOF condensation, generalized eigenvalue solution, mode-shape recovery, participation factors, effective modal masses, and modal response parameters. This validates the modal solver itself before applying it to larger generated frames.

This validation confirmed that the computed natural frequencies and modal response quantities matched the lecture examples within numerical roundoff. The validation cases also confirmed that the solver correctly handles systems where not all free degrees of freedom have assigned mass. In such cases, massless DOFs are condensed before solving the eigenproblem, and the full mode shapes are then recovered for display and postprocessing.

6.4 Robot Five-Story Frame Validation

The generated five-story unbraced frame was also checked against Autodesk Robot for modal analysis. The Robot model used the same geometry, support conditions, material stiffness, section properties, and lumped lateral nodal masses as the Python model. The companion mass file assigns 50,000kg in the lateral translational direction at each free floor node, giving a total lateral modal mass of 500,000kg.

Table 2. Robot modal validation for the five-story unbraced frame

Mode	Python frequency (Hz)	Robot frequency (Hz)	Result
1	1.1686	1.17	Close agreement
2	3.6282	3.63	Close agreement
3	6.3691	6.37	Close agreement
4	9.0213	9.02	Close agreement

This comparison validates the generated-frame modal workflow, not only the standalone eigenvalue solver. Since eigenvector normalization is not unique, the comparison focused on frequencies, periods, mode-shape patterns, and participating-mass behavior rather than raw mode-shape signs or unnormalized participation factors.

6.5 Braced/Unbraced Generated-Frame Check

The generated ten-story unbraced and braced frames were used as engineering trend-validation cases. These examples check whether the complete generator–solver–postprocessor workflow produces physically reasonable behavior for larger models.

Table 3. Braced/unbraced generated-frame comparison

Quantity		10-story unbraced	10-story braced	Change
Roof displacement (u_x)	(mm)	5.708742	4.181666	26.75% reduction
Maximum story drift	(mm)	0.8830496	0.6270207	28.99% reduction
First natural frequency	(Hz)	0.748922	0.878754	17.34% increase

The observed trend is structurally reasonable: adding braces increases lateral stiffness, which reduces roof displacement and story drift while increasing the first natural frequency. Overall, the verification evidence supports two conclusions. First, the structural backend is stable for the implemented two-dimensional linear frame–truss scope. Second, the expanded GUI, input/output workflow, postprocessing tools, and generated-frame examples call the same verified computational engine and produce physically consistent results

7. Error Handling and User Safety

The program includes several safeguards for common user and modeling errors. Missing or malformed model files are caught by the loader and reported through GUI error dialogs, while invalid text-deck syntax is reported with information such as the line number, keyword, missing field, or invalid reference. Duplicate IDs, invalid references, unsupported member-load definitions, and invalid truss-offset cases are rejected during model construction or GUI validation. During analysis, unstable or singular static models stop with mechanism- or singularity-oriented error messages instead of returning misleading results. Modal analysis similarly rejects invalid massive-DOF cases, uses documented fallback logic only where appropriate, and stops if the massless-DOF condensation block is singular. If any analysis fails, stale result objects are cleared before the error is reported, reducing the risk of displaying outdated results.

8. Limitations and Future Work

The software is a linear elastic two-dimensional frame–truss educational analysis package. It is not a full commercial structural design program. The current implementation does not include full 3D frame behavior, shell or slab elements, rigid diaphragm constraints, nonlinear material

behavior, geometric nonlinearity, P-Delta effects, staged construction, design-code checks, or commercial load-combination workflows.

Several implemented features should be interpreted as controlled extensions. Axis offsets are limited to 2D frame local-y offsets, and member loads are limited to supported point loads and full-span uniformly distributed loads. Hermite deformed-shape plotting, station-force recovery, linear modal RHA, and RSA postprocessing are experimental educational add-ons. They improve interpretation and demonstration, but they are not substitutes for a full FEM/FEA formulation, nonlinear response-history analysis, or a complete seismic design workflow.

Future work may extend the current Direct Stiffness Method solver into a broader FEM/FEA platform. Possible developments include 3D frame elements, shell/slab elements, rigid diaphragm constraints, partial-span member-load formulas, P-Delta analysis, nonlinear response-history analysis, design-code checks, and improved CAD-style model editing. A longer-term extension could add reusable shape-function libraries, Gauss quadrature, isoparametric mapping, higher-order elements, mesh generation, and contour visualization.

9. Conclusion

This project extends the earlier CE4011 Direct Stiffness Method solver into a GUI-based 2D frame-truss analysis program with linear static analysis, modal analysis, postprocessing, and verification tools. The main objective was satisfied by implementing lumped mass assembly, massless DOF condensation, generalized eigenvalue analysis, mode-shape recovery, modal participation factors, effective modal masses, and graphical modal postprocessing, while also exposing earlier assignment features such as thermal loads, support settlements, member-force recovery, and static visualization. Additional controlled extensions include generated frame models, member loads, axis offsets, Hermite/station postprocessing, drift and roof-displacement output, solver diagnostics, unit metadata, and experimental modal-response utilities. The final implementation passed 347 automated tests and 13 out of 13 GUI backend verification cases, demonstrating a complete educational workflow for model definition, static/modal solution, visualization, validation, and documentation within a linear elastic 2D scope. Furthermore, modal behavior was validated both through CE586 benchmark examples and through an Autodesk Robot comparison of the generated five-story frame.

References

1. Kurç, Ö. (2026). *CE4011 term project guidelines* [Course project handout]. Department of Civil Engineering, Middle East Technical University.
2. Kurç, Ö. (2026). *CE4011 assignments 1–4: Project materials* [Course materials]. Department of Civil Engineering, Middle East Technical University.
3. Kurç, Ö. (2026). *CE4011 course notes: Direct stiffness method and structural analysis* [Lecture notes]. Department of Civil Engineering, Middle East Technical University.
4. Erberik, M. A. (2026). *CE586 lecture notes #6: Modal analysis* [Lecture notes]. Department of Civil Engineering, Middle East Technical University.
5. Python Software Foundation. (2026). *Python 3 documentation*. <https://docs.python.org/3/>
6. Python Software Foundation. (2026). *tkinter — Python interface to Tcl/Tk*. In *Python 3 documentation*. <https://docs.python.org/3/library/tkinter.html>
7. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
8. Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
9. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
10. OpenAI. (2026). *ChatGPT* (GPT-5.5 Thinking, June 19 version) [Large language model]. <https://chat.openai.com/>
11. Anthropic. (2026). *Claude* (Claude 4.6 Sonnet) [Large language model]. <https://claude.ai/>

OpenAI ChatGPT and Anthropic Claude were used as AI-assisted planning, drafting, debugging, and documentation-support tools. Final engineering decisions, implementation, testing, verification, and submitted content remain the responsibility of the author.

Appendix

Appendix A - Static Verification

A.1 Cantilever beam with end load

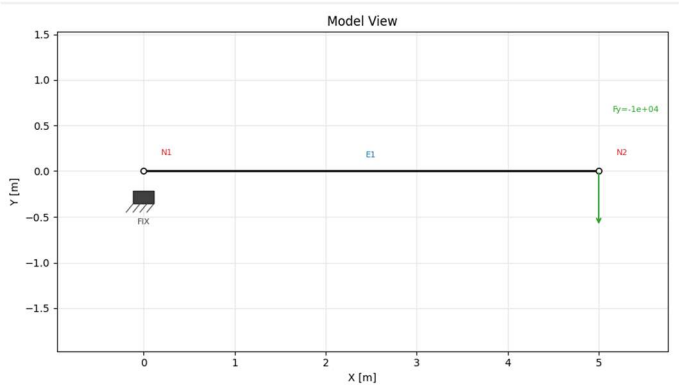


Figure A1.1. Cantilever Beam Model

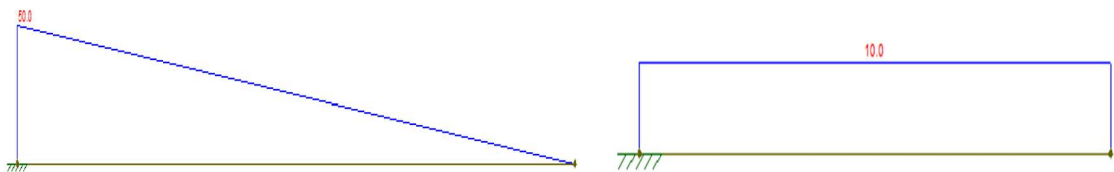


Figure A1.2. Ftool Cantilever SFD/BMD

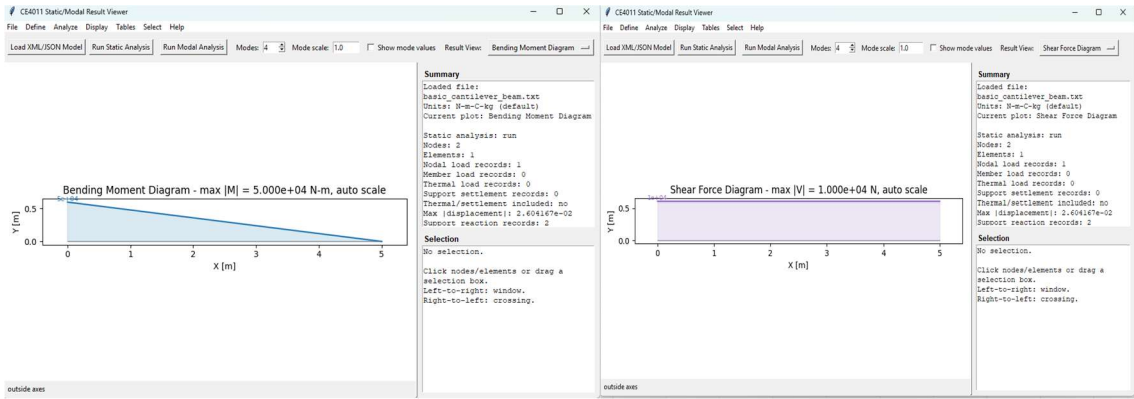


Figure A1.3. Solver Cantilever SFD/BMD

Table A1. Ftool-Solver SFD/BMD Values comparison

Value	Solver	Ftool	%Difference
$V_{\max}$ (kN)	10.0	10.0	0.00
$M_{\max}$ (kNm)	50.0	50.0	0.00

A.2Simply supported beam with UDL load

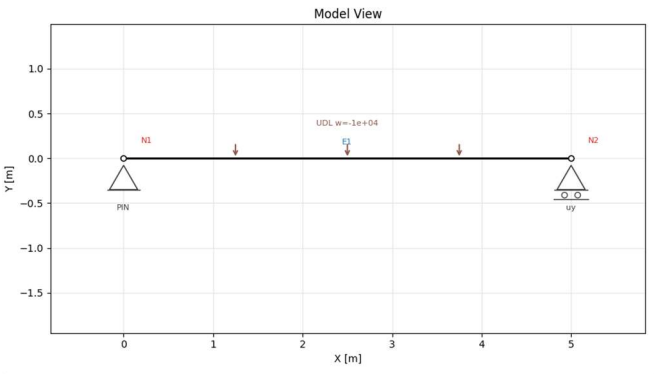


Figure A2.1. Simply Supported Beam Model

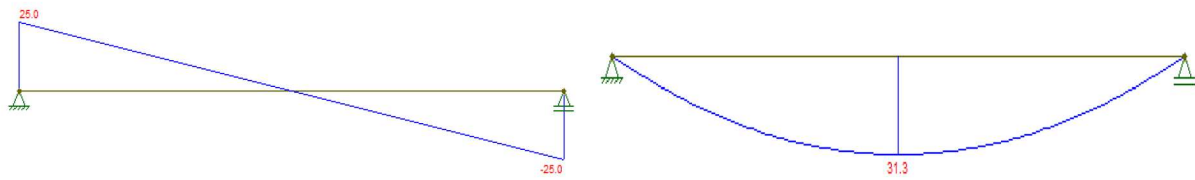


Figure A2.2. Ftool Simply Supported SFD/BMD

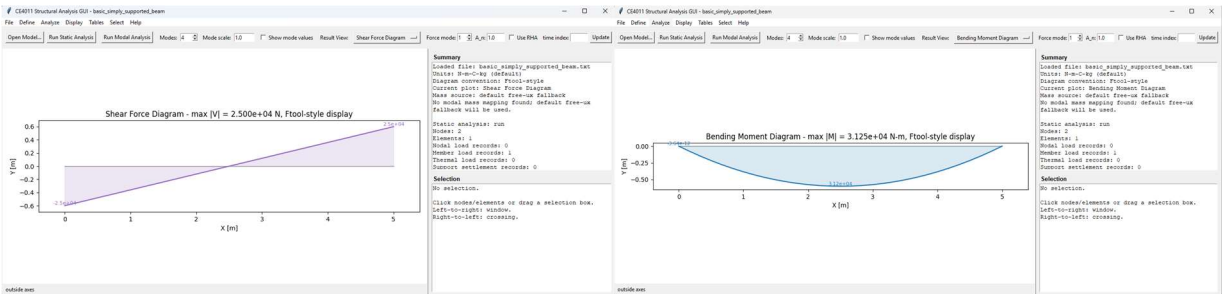


Figure A2.3. Solver Simply Supported SFD/BMD

Note: SFD is inverted due to opposing sign conventions, otherwise the magnitudes remain the same.

Table A2. Ftool-Solver SFD/BMD Simply Supported Values comparison

Value	Solver	Ftool	%Difference
$V_{\max}$ (kN)	25.0	25.0	0.00
$M_{\max}$ (kNm)	31.25	31.25	0.00

Appendix B — Hand-calculation Modal Verification

B.1 CE586 Example 6.2

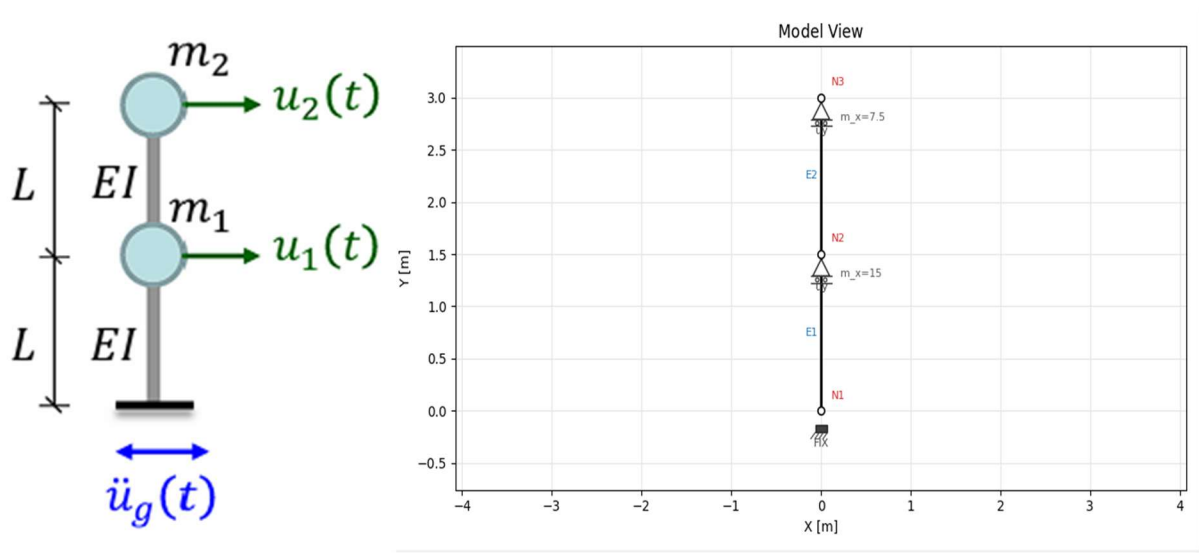


Figure B1.1. 2 story column model

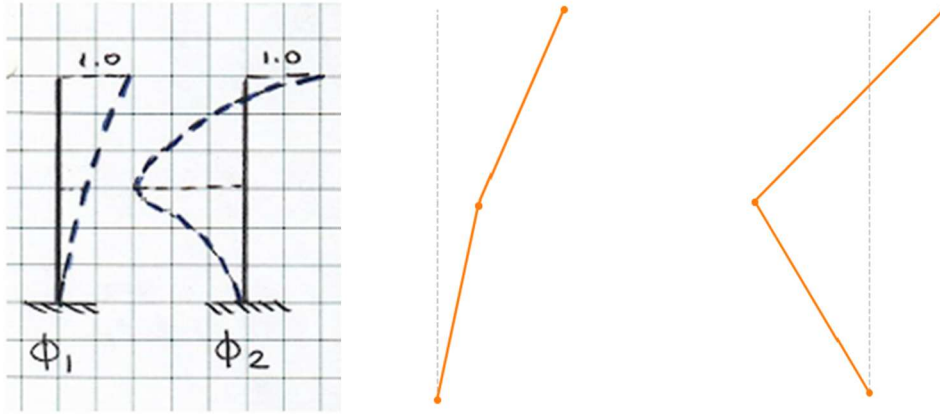


Figure B1.2. Mode 1 and Mode 2 Shape Comparison

Table B1. Modal Properties Comparison

Mode Source ω (rad/s) f (Hz) T (s) Γ_n M_n^* Mass ratio Modal force vector s_n

1	CE586	10.000	1.5915	0.6283	1.36	16.91	0.751	6.67, 10.20
1	Solver	10.1155	1.6099	0.6211	1.3627	16.9112	0.7516	6.6913, 10.2200
2	CE586	52.000	8.2761	0.1208	-0.36	5.59	0.248	8.25, -2.70
2	Solver	52.1060	8.2929	0.1206	-0.3627	5.5888	0.2484	8.3087, -2.7200

B.2 CE586 Example 6.4

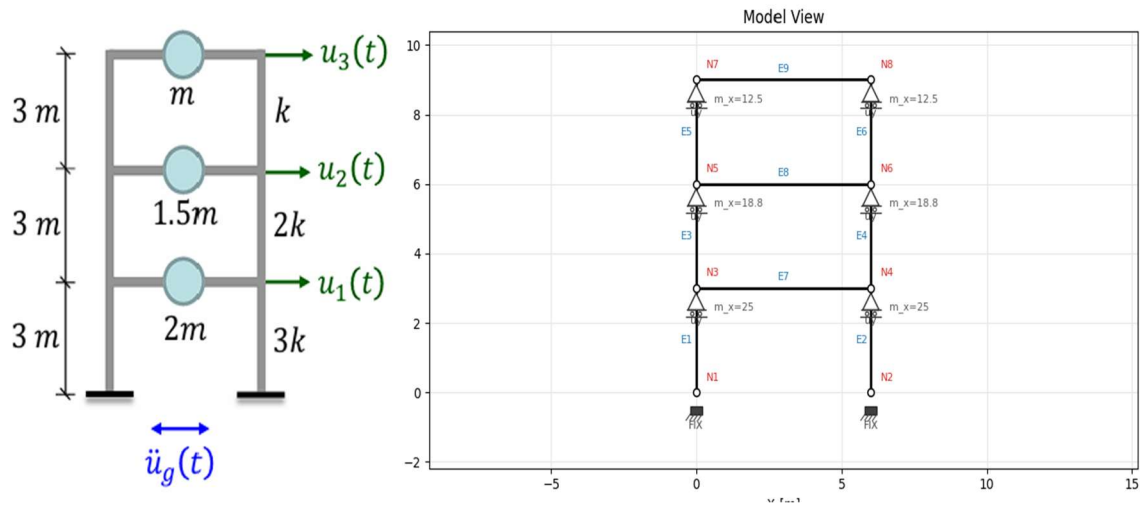


Figure B2.1. 3-storey 2-column frame model

Table B2. Modal Properties Comparison

Mode	Source	T (s)	Γ_n	M_n^*	h_n^*	Modal force vector, s_n	V_{bn}	M_{bn}
1	CE586	1.53	1.421	91.540	6.461	21.457, 34.530, 35.525	91.540	591.440
1	Solver	1.529946	1.421008	91.532	6.461	21.446, 34.561, 35.525	91.532	591.429
2	CE586	0.72	-0.512	16.229	0.417	17.382, 11.635, -12.800	16.229	6.767
2	Solver	0.715531	-0.51247	16.243	0.420	17.398, 11.657, -12.812	16.243	6.829
3	CE586	0.48	0.0913	4.705	0.361	11.125, -8.703, 2.283	4.705	1.699
3	Solver	0.481888	0.091464	4.725	0.369	11.156, -8.718, 2.287	4.725	1.741

Appendix C — Frame Verification

C.1 Five-story frame static check

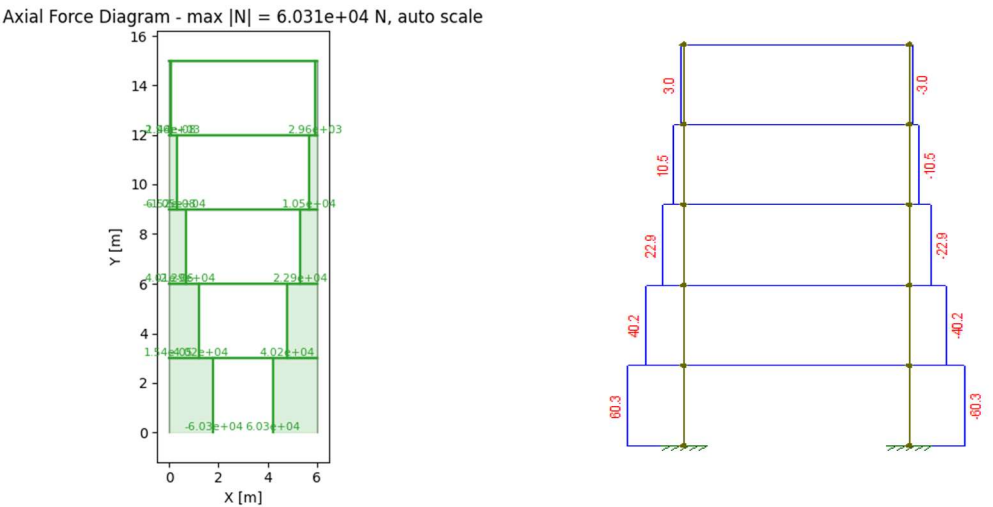


Figure C1.1. 5-storey frame Axial Force Diagram Comparison

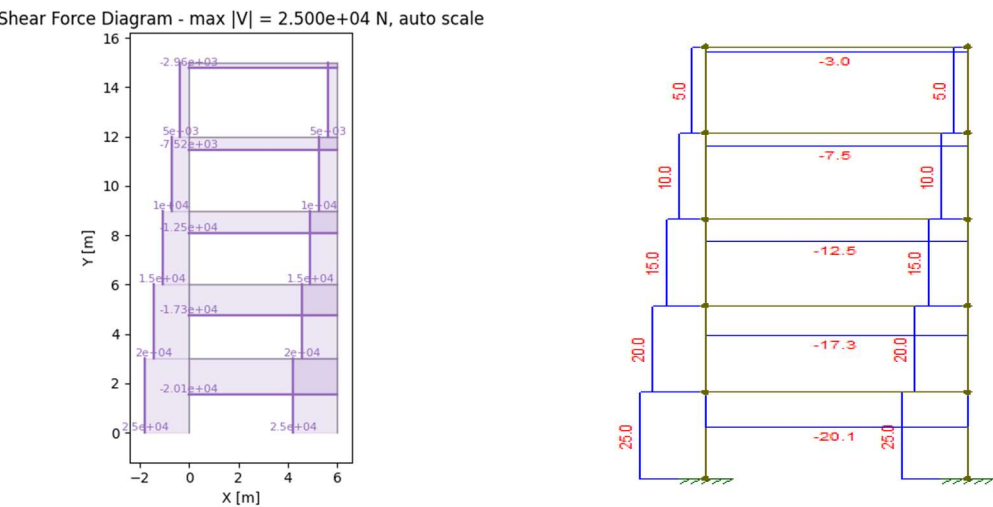


Figure C1.2. 5-storey frame Shear Force Diagram Comparison

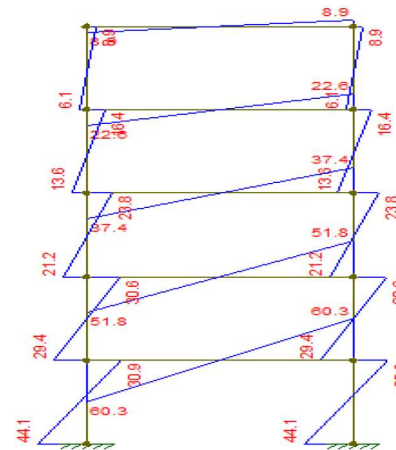
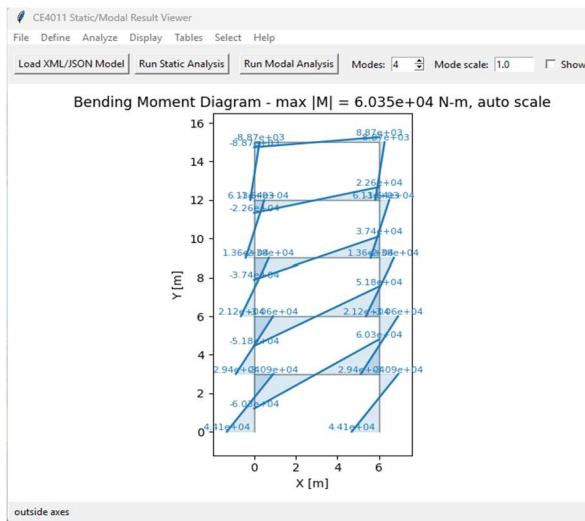


Figure C1.3. 5-storey frame Bending Moment Diagram Comparison

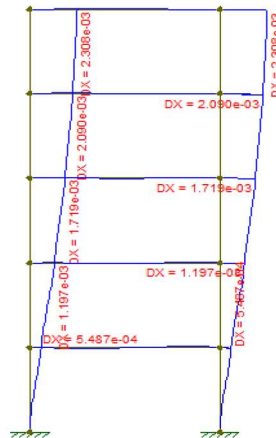


Figure C1.4. 5-storey frame Deformed Shape

C.2 Five-story modal check against Robot

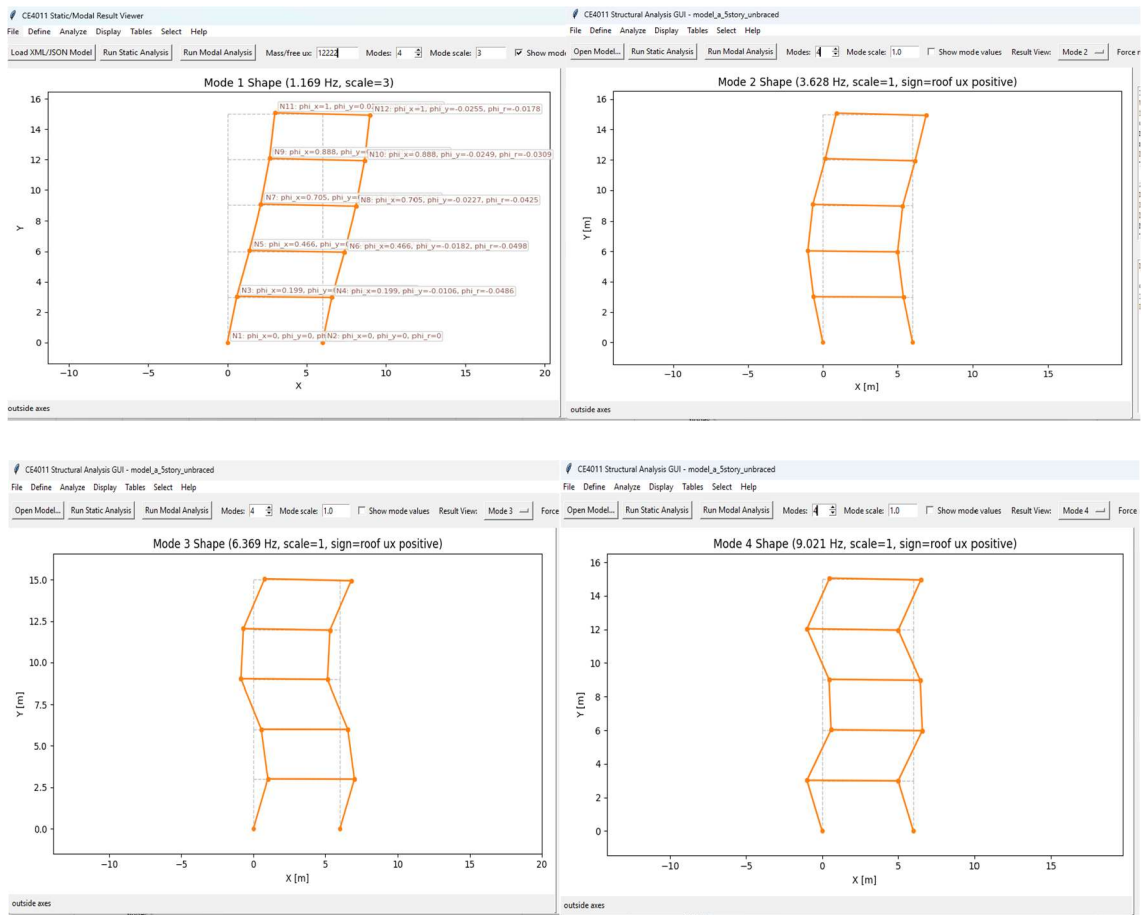


Figure C2.1. 5-story frame mode shapes

Dynamic Analysis Results3 - Case: 1 (Modal) Active modes: 1,4,CQC

Case/Mode	Frequency (Hz)	Period (sec)	Rel.mas.UX (%)	Rel.mas.UZ (%)	Cur.mas.UX (%)	Cur.mas.UZ (%)	Total mass UX (kg)	Total mass UZ (kg)
1/ 1	1.17	0.86	83.50	0.0	83.50	0.0	500170.80	0.0
1/ 2	3.63	0.28	94.38	0.0	10.87	0.0	500170.80	0.0
1/ 3	6.37	0.16	98.04	0.0	3.66	0.0	500170.80	0.0
1/ 4	9.02	0.11	99.58	0.0	1.55	0.0	500170.80	0.0

Figure C2.2. 5-story Autodesk Robot Modal Analysis

Table C2.1 Autodesk Robot and Solver Modal Frequencies Comparision

Mode	Solver (ω) (rad/s)	Solver Frequency (Hz)	Solver Period (T)	Robot Frequency (Hz)	Robot Period (T)
1	7.342351	1.168571	0.855746	1.17	0.86
2	22.79634	3.628150	0.275623	3.63	0.28
3	40.01822	6.369098	0.157008	6.37	0.16
4	56.68265	9.021323	0.110849	9.02	0.11

Table C2.2 Modal Analysis Summary

Mode	ω rad/s	f Hz	T_s	Γ_n	M_n^* (kg)	Mass ratio	V_{bn} (kN)	M_{bn} (kNm)
1	7.342	1.1686	0.8557	1.2816	417,513	83.50%	417.513	4,535.811
2	22.796	3.6282	0.2756	-0.4085	54,370	10.87%	54.370	-81.009
3	40.018	6.3691	0.1570	0.1877	18,301	3.66%	18.301	42.469
4	56.683	9.0213	0.1108	-0.0817	7,731	1.55%	7.731	-0.250

C.3 Unbraced ten-story modal check

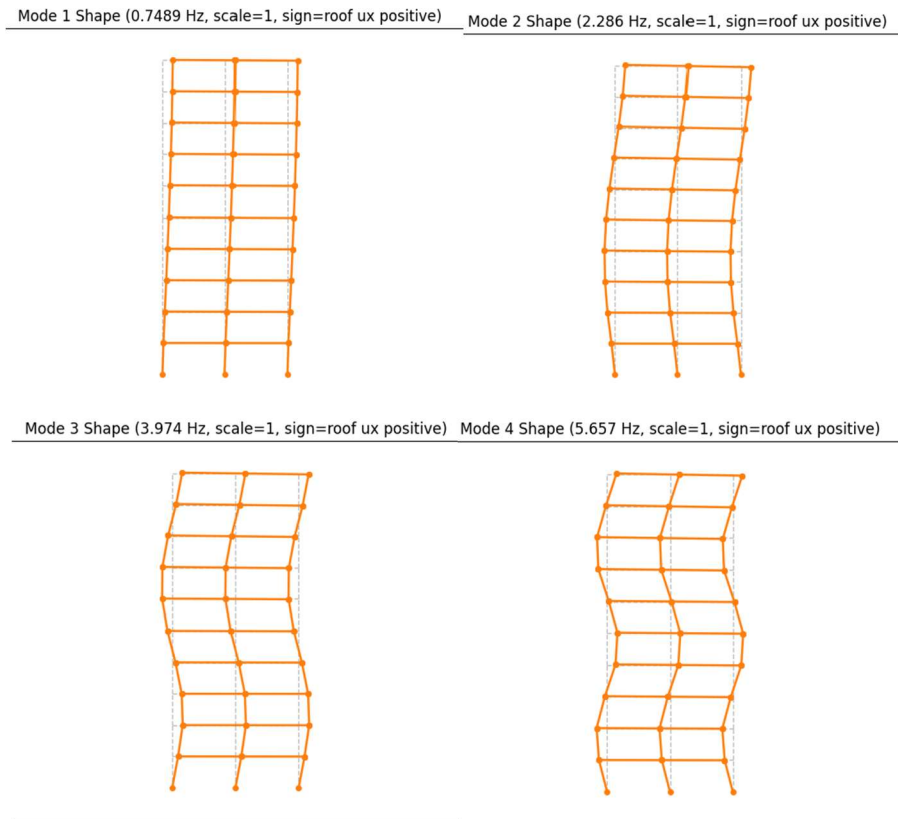


Figure C3.1. 10-story unbraced frame mode shapes

Table C3.1 Modal Analysis Summary

Mode	ω rad/s	f Hz	T s	Γ_n	M_n^* (kg)	Mass ratio	V_{bn} (kN)	M_{bn} (kNm)
1	4.7056	0.7489	1.3353	1.3062	812,956.80	0.8130	812.956	16,734.7
2	14.3605	2.2855	0.4375	-0.4537	108,030.30	0.1080	108.030	-385.6018
3	24.9711	3.9743	0.2516	0.2421	35,167.11	0.0352	35.1671	138.6587
4	35.5417	5.6566	0.1768	-0.1594	18,056.80	0.0181	18.0568	-19.1070

C.4 Braced ten-story modal check

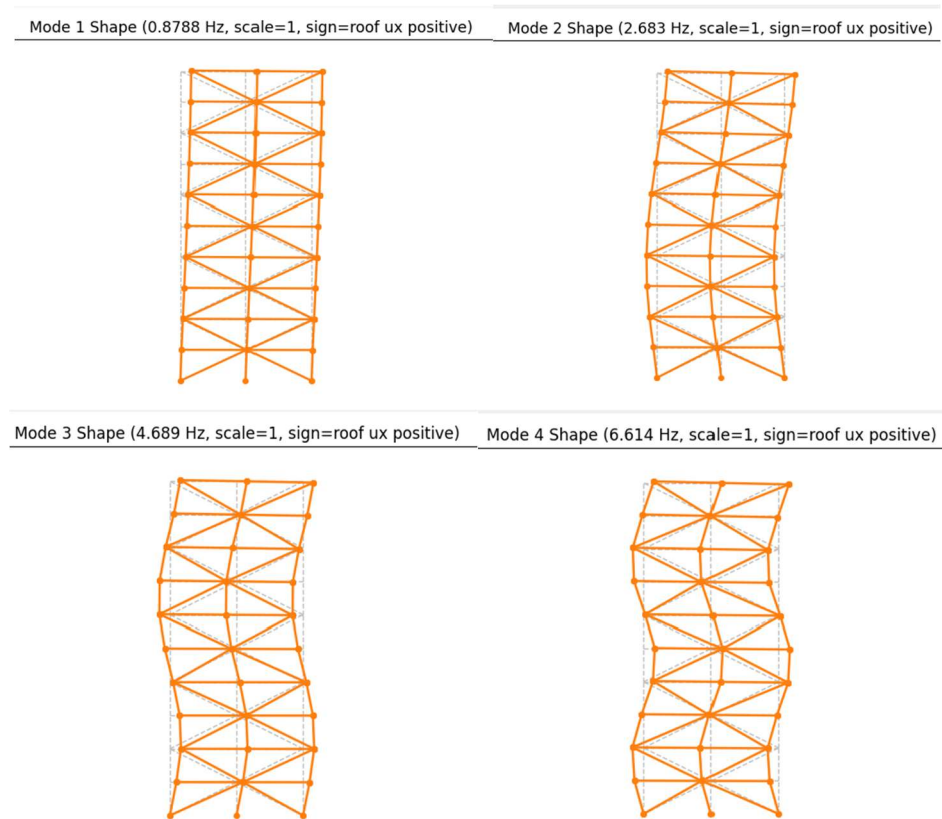


Figure C4.1. 10-story braced frame mode shapes

Table C4.1 Modal Analysis Summary

Mode	ω rad/s	f Hz	T s	Γ_n	M_n^* (kg)	Mass ratio	V_{bn} (kN)	M_{bn} (kNm)
1	5.521	0.8788	1.1380	1.31	810,400	81.04%	810.400	16,717.920
2	16.860	2.6834	0.3727	-0.4652	113,833	11.38%	113.833	-351.966
3	29.459	4.6886	0.2133	0.2391	34,798	3.48%	34.798	126.249
4	41.558	6.6142	0.1512	-0.1551	17,396	1.74%	17.396	-18.240

Appendix D — Experimental Dynamic Add-ons

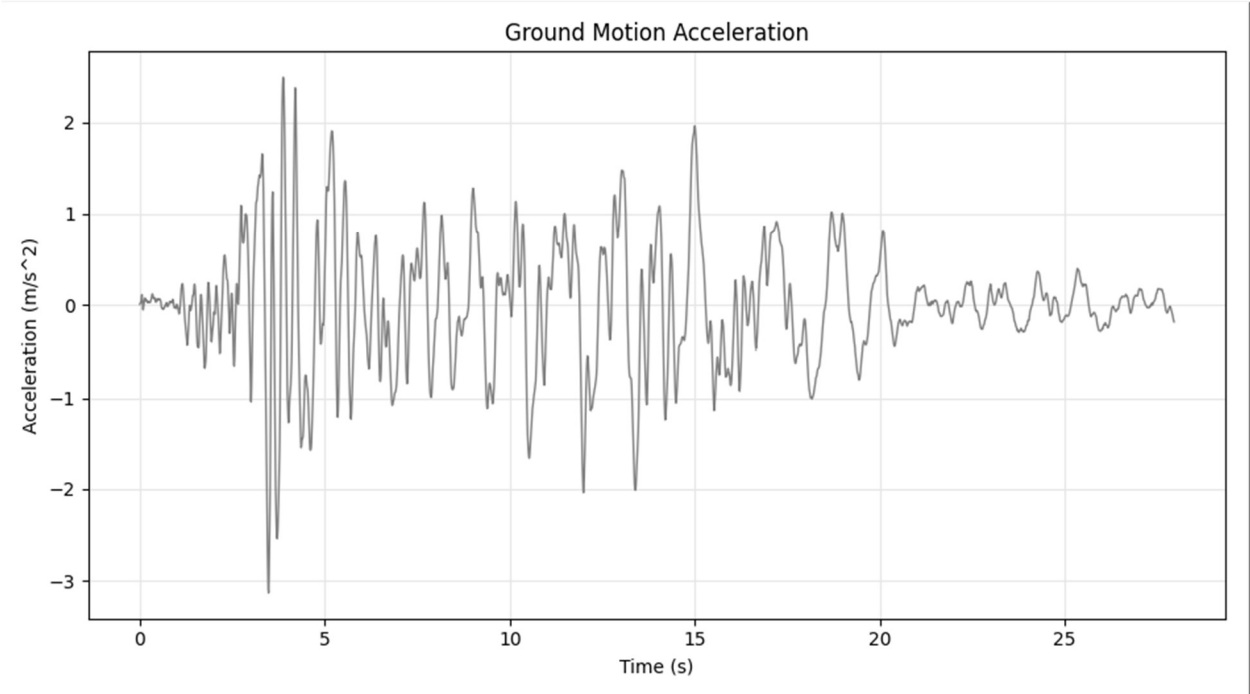


Figure D1. Ground Motion Acceleration

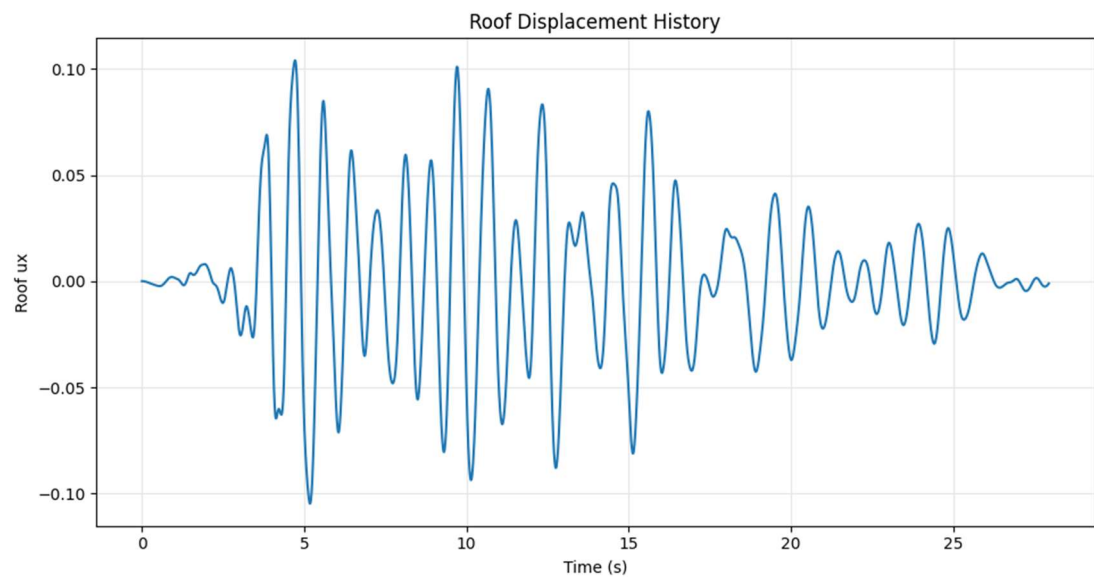


Figure D2. Roof Displacement History for 5-story model

Table D1. ABSSUM, SRSS and CQC of RSA on 5-story model

Quantity	Location	ABSSUM	SRSS	CQC
Floor displacement u	Floor 1	2.91E-02	2.28E-02	2.28E-02
Floor force f	Floor 1	6.48E+05	3.74E+05	3.81E+05
Floor displacement u	Floor 2	6.17E-02	5.21E-02	5.22E-02
Floor force f	Floor 2	9.11E+05	5.93E+05	5.95E+05
Floor displacement u	Floor 3	8.49E-02	7.77E-02	7.78E-02
Floor force f	Floor 3	9.15E+05	5.55E+05	5.54E+05
Floor displacement u	Floor 4	1.00E-01	9.76E-02	9.76E-02
Floor force f	Floor 4	7.82E+05	5.47E+05	5.46E+05
Floor displacement u	Floor 5	1.20E-01	1.10E-01	1.10E-01
Floor force f	Floor 5	1.20E+06	7.64E+05	7.60E+05
Base shear Vb	Global	2.69E+06	2.03E+06	2.04E+06
Base moment Mb	Global	2.22E+07	2.10E+07	2.10E+07

Note: All SRSS and CQC values are almost identical. This is because the modal frequencies are well-separated. Both values are significantly less than ABSSUM, as ABSSUM is more conservative.

Appendix E — Miscellaneous

```

PS C:\DDrive\Imran\METU\Coursework\CE4011\Project> C:\Users\Imran\anaconda3\envs\CE4011\python.exe -m pytest -q
..... [ 34%]
..... [ 68%]
..... [100%]
347 passed in 6.52s
PS C:\DDrive\Imran\METU\Coursework\CE4011\Project> C:\Users\Imran\anaconda3\envs\CE4011\python.exe scripts\run_gui_backend_verification.py
GUI backend verification summary

=====
Case ID                                Expected  Status  Message
-----
A4_Q2_MODEL_A                         solve     PASS    nodes=6, elements=5, reactions=6, member_forces=5
A4_Q2_MODEL_B                         solve     PASS    nodes=5, elements=4, reactions=5, member_forces=4
A4_REG_THERMAL_TRUSS_XML               solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_REG_THERMAL_GRADIENT_XML           solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_REG_THERMAL_COMBINED_XML           solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_REG_SETTLEMENT_XML                 solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_JSON_THERMAL_BAR                   solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_JSON_THERMAL_BEAM_GRADIENT         solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_JSON_THERMAL_FRAME_COMBINED        solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
TEXT_SIMPLE_PORTAL                   solve     PASS    nodes=4, elements=3, reactions=4, member_forces=3
A4_TEXT_SETTLEMENT_EXAMPLE            solve     PASS    nodes=2, elements=1, reactions=2, member_forces=1
A4_TEXT_THERMAL_EXAMPLE               solve     PASS    nodes=5, elements=4, reactions=5, member_forces=4
LEGACY_SETTLEMENT_JSON                error     PASS    Expected error: AttributeError: 'list' object has no attribute 'get'
=====
Total: 13, PASS: 13, FAIL: 0
Plot output root: C:\DDrive\Imran\METU\Coursework\CE4011\Project\results\verification\gui_backend
PS C:\DDrive\Imran\METU\Coursework\CE4011\Project> C:\Users\Imran\anaconda3\envs\CE4011\python.exe scripts\run_final_core_smoke.py
Final core smoke summary

=====
Check                                Status  Message
-----
load generated model                 PASS    model_a_5story_unbraced.json
static nodal displacements           PASS
static support reactions             PASS
static member-end forces             PASS
static/core plots saved              PASS    C:\DDrive\Imran\METU\Coursework\CE4011\Project\results\final_core_smoke
modal frequencies                    PASS
modal periods                        PASS
mode-shape data                      PASS
modal participation                  PASS
modal plots saved                    PASS    C:\DDrive\Imran\METU\Coursework\CE4011\Project\results\final_core_smoke
story drift rows                     PASS
roof displacement                    PASS

```

Figure E1. Code testing snippet of verification

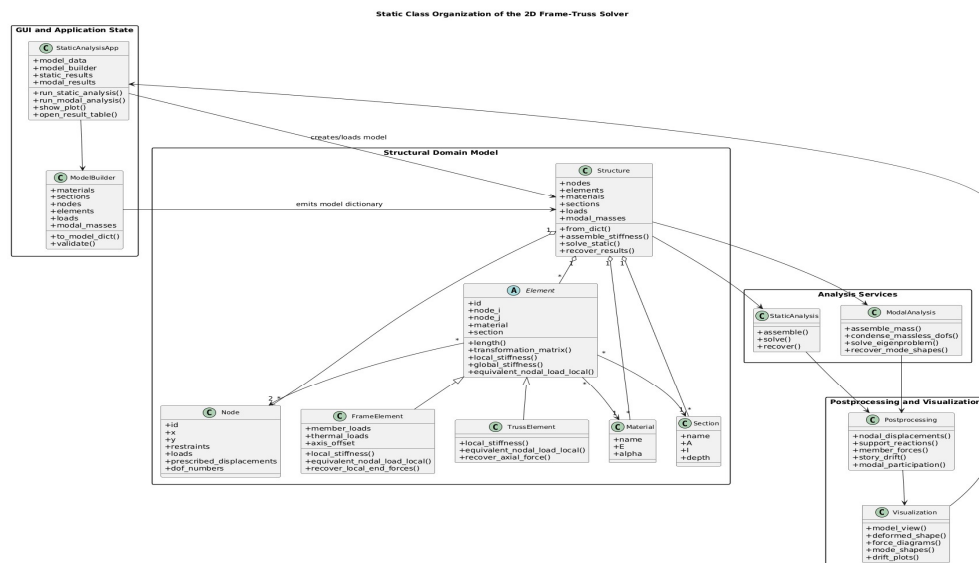


Figure E2. Full UML Diagram

Note: Higher resolution UML Diagrams are present in the Google Drive Link